

School LLM

Technical White Paper on the AI / RAG Architecture

1. Overview

The School LLM is a retrieval-augmented learning platform. Students upload textbook PDFs and the system answers questions, generates summaries and quizzes, and grades assignments. This paper concentrates on the three engineering areas the AI architecture actually depends on: the RAG pipeline, the vector database and prompt engineering layer, and the hybrid Claude-plus-Ollama strategy at the model layer.

Technology stack

Layer	Choice	Role
PDF extraction	PyMuPDF → PyPDF2 → Tesseract OCR	Three-tier strategy; OCR triggered when extracted text is < 50 chars
Tokenizer	tiktoken (gpt-3.5-turbo encoding)	Stable cross-provider token counts
Embedding model	sentence-transformers/all-MiniLM-L6-v2 (384-dim, L2-normalized)	Ships with project, runs on CPU
Embedding alternative	Ollama nomic-embed-text	Single-stack deployments
Vector database	ChromaDB (PersistentClient, HNSW, cosine)	On-disk, no managed dependency
LLM (production)	Anthropic Claude — Sonnet 4.6 + Haiku 4.5, tiered	Highest-quality student-facing output
LLM (dev / offline / fallback)	Ollama with qwen2.5:3b	Free, offline-capable
Provider abstraction	Custom LLMClient Protocol with three implementations	Provider-agnostic call sites
Resilience	FallbackLLMClient with 60s per-provider cooldown	Skips dead providers without burning timeouts
Backend / API	FastAPI	Async-first
Application database	MongoDB Atlas	Users, sessions, assignments, audit
Frontend	Streamlit	Single-language UI

2. The RAG Pipeline

2.1 PDF Text Extraction — Three Tiers

A real school PDF library is heterogeneous: born-digital textbooks, phone-camera scans of worksheets, multi-column exam papers. During development, a single extraction library failed on roughly one document in five, which led to the three-tier design.

Tier 1 — PyMuPDF. Used as the primary extractor. The dictionary-form call `page.get_text("dict")` preserves block / line / span structure with bounding boxes and font-size metadata. This matters because an early version using plain text-mode silently flattened **H₂O** to **H2O** and **x²** to **x2** — those embed as different tokens and quizzes marked correct answers wrong. The dict form lets the extractor walk each span and rebuild the original notation as plain text (**H₂O**, **x²**) using font-size deltas and vertical-position analysis.

Tier 2 — PyPDF2. Used when PyMuPDF fails to open the file (broken object streams, missing fonts). Loses formatting detail but tolerates malformed PDFs.

Tier 3 — Tesseract OCR. Activated per-page when extracted text is under 50 characters — the signal for an image-only page. Pages are rasterized at 200 DPI; 150 DPI showed visible recognition errors during testing, 300 DPI tripled ingestion time without measurable accuracy gain.

Normalization. Every extracted line passes through a Unicode normalizer that unifies non-breaking spaces, three dash variants, smart quotes, and pre-punctuation whitespace. This was added after observing that **photo-synthesis** written with a soft hyphen and **photosynthesis** written normally were embedding differently, with no error to flag the recall drift.

Structural metadata. Regex captures section codes (**2.1.3**, **Exercise 1.5**), section titles, parent-section breadcrumbs, and content type (text, exercise, worked example) at extraction time. This metadata feeds the reranker and citation tags downstream.

Code: `backend/pdf_handler.py`.

2.2 Chunking Strategy

LangChain's **RecursiveCharacterTextSplitter** was the first chunker tried. On textbook PDFs it cut across section boundaries and split worked examples, which the reranker could not fully recover from. The custom chunker that replaced it is paragraph-first and heading-aware: accumulate paragraphs (double-newline split) up to a 750-token budget with 100-token overlap, and force a new chunk on any line matching a section-heading or exercise-heading regex.

Token counting uses **tiktoken** with the gpt-3.5-turbo encoding for stable cross-provider counts. If tiktoken fails to import, the fallback is `len(text) // 4`, which slightly overestimates and so never produces an oversized chunk.

Why 750 / 100. A 750-token chunk almost always holds a complete worked example or definition + example pair. The embedder's effective input window is around 256 tokens after pooling, so chunks get truncated — but the truncated content is rarely critical for capturing the chunk's topic, while a smaller chunk would split worked examples and force the retriever to stitch fragments. 200-token overlap was tested and produced duplicate retrievals the reranker had to clean up; 100 was the smallest overlap that preserved recall without polluting the candidate pool.

Each chunk carries **chunk_id**, **text**, **token_count**, and a metadata dict (**page_number**, **chapter**, **section_code**, **section_title**, **topic**, **headers**, **content_type**).

The study_context distillation. A document-wide context of up to 8000 characters is pre-computed by sampling the first 12 sections plus chunks from beginning / middle / end. Summary and quiz generation consume this distillation rather than raw text. On a 300-page textbook this cut Ollama summary latency from ~25s to ~6s — the largest single latency win in the system.

2.3 Embedding Generation

Model: `sentence-transformers/all-MiniLM-L6-v2` — 384-dimensional, L2-normalized.
nomic-embed-text via Ollama is an alternative for single-stack deployments.

bge-small and **e5-small-v2** were both tested first. On a development-time evaluation set built from ~200 real student questions, MiniLM-L6 outperformed both on the imprecise structural queries that turned out to be the majority of real traffic ("tell me about chapter 3", "explain exercise 2.1.5").

bge-small scored slightly better on direct factual queries but worse on structural ones, and the structural failure cases were the more painful ones to debug.

The other two reasons MiniLM-L6 stayed: 384 dimensions keep ChromaDB lookups sub-millisecond on CPU, and the ~80 MB on-disk footprint ships inside the project with no first-launch download phase — meaningful for offline school deployments.

L2-normalization is mandatory. With normalized vectors, cosine similarity reduces to a plain dot product, and reranker scores stay comparable across queries — a half-point boost means the same thing regardless of question length. The grounding check (Section 3.9) became unstable when normalization was briefly disabled during testing.

The synchronous **encode()** is wrapped in **asyncio.to_thread** so it doesn't block the FastAPI event loop. The Q&A pipeline rewrites a question into up to four retrieval variants and embeds all four in a single batched call — saves ~250 ms per request on CPU.

2.4 Retrieval Flow

Retrieval is two-pass. Pass one pulls 3k chunks ranked by cosine distance — a wider net than the final top-k. Pass two reranks with metadata-aware boosts. Pulling 3k gives the reranker room to recover when the embedder ranks the right chunk fourth or fifth instead of first, which happens often enough to matter.

A cross-encoder reranker (**bge-reranker-base**, Cohere Rerank) was the conventional next step, but it was rejected for two reasons: each candidate needs a forward pass through the reranker model (roughly doubling per-query CPU cost on a CPU-only school machine), and cross-encoders cannot see structural metadata like section codes and breadcrumbs — exactly the signals that distinguish a good match from an irrelevant one on structural queries. The metadata-aware reranker runs in microseconds and uses signals a cross-encoder is blind to.

2.5 Reranker — Boost Weights

base_score = 1 - distance plus additive boosts capped at 1.0:

Signal	Boost	Reasoning
Query term verbatim in chunk (cap +0.30 total)	+0.10 each	Catches mediocre semantic match where the chunk literally contains the words asked about
Query text in chunk's topic metadata	+0.15	Topic is the chunk's local subject from its headings — a topic match is a strong intent signal
Section code in question exactly matches chunk's section_code	+0.25	Strongest structural match. "Explain exercise 2.1.5" should retrieve the 2.1.5 chunk.
Chunk's section in caller's preferred_section_codes (conversation history)	+0.35	Highest single boost. Keeps multi-turn conversations on topic.
Chunk's section is parent or child of a preferred section	+0.20	Picks up follow-ups that drift to an adjacent section
Query term in chunk's section_title	+0.15	Section title is a high-signal summary of chunk content

The 1.0 cap exists so structural matches can't completely override semantic relevance — a chunk in the right section on a clearly different sub-topic still has to clear a semantic threshold.

Weights were tuned by capturing ~200 real student questions during early development, hand-labelling the correct chunks for each, and iterating until both structural and open-ended subsets improved together. They aren't from a paper.

3. Vector Database and Prompt Engineering

3.1 Why ChromaDB

Pinecone, Qdrant, Weaviate, and pgvector were evaluated against the deployment profile:

1. Must run on a single mid-range machine in a school IT closet. No managed cloud.
2. Must survive a power cycle without re-indexing. Re-indexing every restart is not viable.
3. Must be inspectable by someone who is not an infrastructure engineer.
4. Dependency footprint must stay minimal — adding Postgres just for vector search was hard to justify when no other application data lived there.

ChromaDB's **PersistentClient** is on-disk, embedded, free, and single-import. Pinecone failed constraints 1 and 4. Qdrant and Weaviate require separate server processes. pgvector required adding Postgres alongside MongoDB. ChromaDB was the simplest option that satisfied all four.

HNSW with cosine space matches the L2-normalized embeddings. HNSW parameters use Chroma's defaults — manual tuning gave no measurable improvement on per-PDF collections of the size this product produces.

3.2 Indexing Strategy — One Collection per PDF

Two organizational patterns are common: shared collection with PDF-id metadata filter, or per-PDF collections. This system uses per-PDF.

Reason. HNSW query latency grows with collection size. A shared collection has every search navigating through every chunk of every PDF; per-PDF collections cap each search at a few thousand chunks. Performance stays flat as the library scales from 10 PDFs to 10,000.

Collection naming. Fixed `pdf_` prefix plus an MD5 hash combining a schema-version tag (currently `2026-03-31-section-chunks-v1`) with the PDF URL. Including the schema version in the hash means bumping the version on a chunking-strategy change produces a fresh collection automatically — no migration script, no chance of old vectors silently corrupting retrieval after a chunker change. This was added after a chunking experiment briefly mixed old and new vectors and the quality drop was hard to attribute.

Idempotent indexing. Before inserting documents, `collection.count() > 0` short-circuits re-embedding when the PDF is already indexed. In a classroom where the same chapter PDF is opened by twenty students in the same day, this turns a 40-second indexing operation into a no-op for nineteen of them.

3.3 Retrieval Optimization

Three things contribute beyond the boost weights:

- **Query-side batching.** Four retrieval variants embedded in one batched `encode()` call instead of four sequential calls (~250 ms saved on CPU).
- **Conversation-derived preferred sections.** Section codes from chunks used earlier in the conversation get passed back in as `preferred_section_codes`. Single signal that makes follow-ups like "explain that further" retrieve the right chunks without restating the topic.
- **Pre-LLM refusal.** The grounding check (Section 3.9) drops weakly-matched questions before any LLM call. Cheapest possible optimization.

3.4 Prompt Engineering Methodology

Three principles that hold across every prompt in the system, stated once and assumed throughout the rest of this section:

1. **Prohibitions outperform positive instructions on smaller models.** "Do X" gets ignored by qwen2.5:3b about a third of the time on test traffic. "NEVER do Y" gets followed almost always. Small models copy structural cues from the user message into the answer unless explicitly forbidden.
2. **Layer dynamic instruction over a static base.** Static base contains rules and prohibitions; per-request layers (intent, audience, history) are appended at runtime. Keeps the base cacheable (Section 4.4).
3. **Validate the output, don't trust it.** Every generation passes through a validator. Failed outputs degrade visibly (a fixed refusal sentence, a dropped question) rather than invisibly (a hallucination, a silently wrong quiz).

3.5 Q&A Prompt Design

Static prompt opens with role + seven prohibitions:

1. Never repeat source passages verbatim — paraphrase.

2. Never use the words **Question**, **Evidence**, **Source**, **Task**, **PDF**, **Context Hint** as headings.
3. Never copy the structure of the prompt into the answer.
4. Use only facts from the supplied passages.
5. For out-of-document questions, return a single fixed sentence. Earlier versions used model-phrased refusals and they leaked ungrounded knowledge ("I can't answer from the PDF, but generally..."). A fixed sentence removes the failure mode entirely.
6. For math, plain text only (x^2 , $\sqrt{x}$) — Streamlit can't render LaTeX without MathJax.
7. Don't summarize at the end. Don't add "I hope this helps."

Intent layer. Classifier returns one of seven intents (**direct_qa**, **problem_solving**, **section_explanation**, **follow_up**, **summary**, **chapter_list**, **overview**); each injects a structural instruction (problem-solving → numbered steps + bolded final answer, chapter listing → numbered list with one-sentence descriptions). Without this layer, the model produced the same shape of answer for every kind of question — particularly visible on chapter-listing queries that came back as long prose.

Audience layer. Admins get 3–5 bullets at 350 tokens; students get 5–9 sentence narrative at 700 tokens. Per-intent bumps: +150 for problem-solving, +80 for explanation/summary.

Context construction. Top 4 reranked chunks, 700 chars each (2800 total). Past ~3000 chars on small models, answer quality drops sharply — model loses the question itself in the noise. Between 2000 and 3000 it's flat. Budget sits near the top of the flat zone. Each chunk prefixed with a citation tag [**S<n>**]**p<page>**]**<section_code>**].

Conversation history. Capped at 4 turns. Longer histories were tested and produced context drift — model latching onto something irrelevant from earlier — that outweighed the benefit.

Code: **backend/ai/qa.py**.

3.6 Summary Generation Logic

A first-try "summarize the following" prompt worked on single-topic articles and failed on everything else: over-focused on chapter one of textbooks, tried to summarize individual exercises on worksheets, gave equal weight to disconnected exam-paper notes.

The current prompt is structurally adaptive — model first decides document type, then picks structure:

Document type	Structure
Multi-chapter textbook	Bullet list, one sentence per chapter, naming each chapter
Single-topic explainer	2–3 short paragraphs
Notes / mixed / exam paper	2–3 paragraphs organized by theme

Letting the model judge document type internally is more reliable than an external heuristic — the model can read the content; a heuristic only sees surface clues.

Two hard rules: preserve every chapter name, formula, term, and proper noun verbatim (so a search-in-summary for "cellular respiration" still hits); and always end on a complete sentence — finish at the next boundary if approaching the token limit. A defensive **_trim_to_last_sentence**

post-processor walks back to the last terminator anyway. Smaller models needed this more often than expected.

Bundled generation does short + detailed in a single JSON-mode call. Output parsed through a three-stage defensive parser (**json.loads** → field-level regex → loose key-value scan). Small models occasionally emit malformed JSON; the parser recovers what it can rather than failing the whole request.

3.7 Quiz Generation Logic

Quiz output is the strictest schema in the system because the frontend renders it into interactive form elements. Bad output here breaks the UI, not just answer quality.

Prompt opens with the literal **Return ONLY a valid JSON object: {"questions": [...]}**, then enumerates fields per question, then gives type-specific schemas:

Type	options	correct_answer	Extra
mcq	dict A/B/C/D	letter	—
true-false	fixed {A: True, B: False}	literal True/False	—
fill-in-blank	empty	missing word/phrase	question must contain _____
short-answer	empty	1–2 sentence model answer	keywords (3–5 terms)
long-answer	empty	3–6 sentence model answer	keywords (5–8 terms)

Keyword-rubric extension. Grading free-text answers without a rubric is unreliable — the AI judge has nothing concrete to compare against. The conventional approach is to run a second LLM call at grading time to derive the rubric from the model answer; that doubles cost and adds latency. Having the generation model emit the rubric in the same call is essentially free since it already knows the key concepts. This was one of the better calls in the system and it surfaced from noticing how much of grading-time work was duplicated effort the generation step had already done.

Mandate clauses. When a specific type is requested, the prompt appends **MANDATORY: every question MUST be MCQ. A question without four options is INVALID and will be REJECTED.** Without it, qwen2.5:3b silently downgrades MCQ to short-answer (easier to generate). A post-generation **_is_valid_for_type** validator drops non-conforming questions — belt and braces.

Focused-RAG mode. When called with a **search_query** and **pdf_identifier**, the module retrieves 8 chunks (850 chars each, 5200 total) for topic-specific quizzes instead of using the full document.

Code: **backend/ai/quiz.py**.

3.8 Output Validation and Parsing

Quiz output goes through five parsing stages:

1. Direct **json.loads** on raw output.
2. Fenced-code-block extraction (``json ...``).

3. Regex for a { ... } block containing the key **questions**.
4. Regex for a flat [...] array.
5. Plain-text fallback — recover questions from numbered lines.

Each stage logs which path engaged. A senior reviewer might reasonably ask why not just enforce strict JSON mode — the answer is that strict JSON mode is supported by Claude and recent Ollama builds but not by every version, and a school site running an older Ollama install still needs to work. The five stages aren't theoretical; each one rescued a real test run during development. They are cheap to add and pay for themselves the first time a model version changes output formatting silently.

3.9 Handling Model Failure Modes

Four characteristic failure modes the system anticipates:

Failure	Mitigation
Weak grounding leading to hallucination	Grounding check runs over five rerank signals (max score, avg score, term-match count, coverage ratio, best per-chunk overlap) before any LLM call. Intent-specific thresholds: chapter-listing at max ≥ 0.45 , explicit section-code at 0.35 (strong on its own), baseline at 0.45 plus length-scaled term-match floor. Failed checks return the fixed refusal sentence with zero LLM calls.
Type drift in quiz generation (MCQ silently downgraded)	Mandate clause + _is_valid_for_type validator drops non-conforming questions before they reach students.
Malformed JSON	Five-stage defensive parser (Section 3.8).
Sentence-mid truncation in summaries	Prompt instructs ending on a complete sentence + _trim_to_last_sentence post-processor walks back if the model ignores it.

The pattern across all four: anticipate the failure, refuse to ship it to the user, degrade visibly rather than invisibly.

4. Hybrid Claude + Ollama Strategy

4.1 The Constraint

A single-provider architecture would have been simpler. Two candidates were considered.

Claude alone: best quality and instruction-following, but costs per token (real at school scale), needs reliable internet (rural schools have intermittent connectivity), has organization-level rate limits that bite when 30 students hit the API together during a homework session.

Ollama alone with qwen2.5:3b: free, offline-capable, no quota, but lower quality on student-facing output (sentence-level coherence drops after a few hundred tokens) and CPU-bound — a single mid-range machine can't serve many concurrent students.

Neither alone was acceptable. The system needs Claude when Claude is available, Ollama when it isn't, and graceful behavior when both have problems.

4.2 Provider Abstraction — LLMClient Protocol

A Python Protocol with a single **chat()** method (OpenAI-style messages-in, string-out). Three implementations: **OllamaWrappedClient**, **AnthropicLLMClient**, **OpenRouterLLMClient**. Every AI module calls the Protocol, never a provider SDK directly.

Protocol chosen over an abstract base class because it gives structural typing without forcing inheritance — new providers can be added by writing a class that implements **chat()**, no registration step, mock implementations for tests stay trivial.

What the abstraction hides:

Concern	Hidden in
System-message extraction (Anthropic separates system from messages)	AnthropicLLMClient
Cache-control attachment (<code>{"type": "ephemeral"}</code>) on system content blocks)	AnthropicLLMClient
HTTP shape, headers, base URLs, API keys	Each client
Streaming vs non-streaming (Ollama <code>/api/chat</code> streams; wrapper consumes the stream)	OllamaWrappedClient
Model-name strings	Exposed as generation_model / evaluation_model / naming_model properties

The OpenRouter rollback (Section 4.8) was painless because of this abstraction — one config change, no edits to the AI modules.

Code: **backend/ai/llm_client.py**.

4.3 Multi-Tier Model Routing Within Claude

Forcing every Claude call onto Sonnet would waste money on low-stakes tasks. Forcing every call onto Haiku would regress quality on student-facing output. Each client exposes three model properties:

Property	Model	Used for	Why this tier
generation_model	claude-sonnet-4-6	Student Q&A, summaries, quizzes	Quality differential matters; 1–2s latency premium acceptable
evaluation_model	claude-haiku-4-5	LLM-as-judge over batches	Per-call quality matters less than throughput; Haiku is ~4× cheaper, scores correlate well with human on the rubrics used
naming_model	claude-haiku-4-5	Auto-generating chat-session titles	Low-stakes utility; Haiku is sufficient

4.4 Prompt Caching

Anthropic's ephemeral prompt caching wraps the static system prompt in a content block with **cache_control = {"type": "ephemeral"}** and sends the request with the **anthropic-beta: prompt-caching-2024-07-31** header. Cache lives ~5 minutes.

The 5-minute window happens to match the duration of a typical evaluation batch. The long static judge rubric (~400 tokens) is paid for once per batch and served from cache for every subsequent item. For student Q&A, the same effect benefits any student asking multiple questions within five minutes.

Empirical impact: 25–50% reduction in evaluation token spend depending on batch size, plus a smaller but measurable latency drop because the cache hit shortcuts input processing. For a 100-teacher / 500-student school running Claude with caching, the order-of-magnitude per-school monthly cost is around USD 700; without caching it's closer to USD 1000. Caching is not optional at production scale.

4.5 Fallback with Cooldown

FallbackLLMClient wraps a primary and secondary client with a per-provider cooldown timestamp. On primary failure, the cooldown is set to **now + 60s** and subsequent requests skip the primary until the cooldown expires.

The failure mode protected against is a sustained outage — IP-level rate limit, regional Anthropic incident, expired key — not a one-off blip. Without a cooldown, every user request would pay a multi-second timeout to the dead provider before retrying the live one. With 30 concurrent students, the system effectively dies even though Ollama is healthy.

Why 60 seconds. 30s was tested first and produced too much retry traffic (the provider was still failing when each retry hit). 120s left the system on Ollama too long after a transient incident recovered. 60s was the operating sweet spot.

Symmetric cooldown on both providers — when both are down, retries don't keep chasing.

4.6 Timeout Handling

Explicit per-call timeouts feed the cooldown — exceeding a timeout is what engages the fallback wrapper:

Operation	Timeout	Reasoning
PDF download	30 s	Most PDFs respond in < 5s; 30s catches slow but recoverable
Ollama chat	60 s	Local network is fast but qwen2.5:3b on CPU can take 20–40s for long answers
Anthropic HTTP	120 s	Generous; failure here is almost always network, not slow generation
OpenRouter HTTP	120 s	Same reasoning

Without explicit timeouts, a hung connection blocks an entire request worker. With them, the fallback has a deterministic point to declare failure.

4.7 Offline Capability via Ollama

The Ollama path is not just a fallback. It's also the development environment, the air-gapped deployment mode, and the cost-controlled mode. Setting **LLM_PROVIDER=ollama** switches every call site to a locally-running Ollama server. Three use cases:

1. **Development.** Running every test through Claude would cost real money.
2. **Air-gapped school deployments.** AI must work without internet.
3. **Cost-controlled deployments.** School's pricing tier doesn't include managed Claude.

Because the **LLMClient** Protocol hides provider differences, no application code changes between the three modes.

4.8 The OpenRouter Experiment (Rolled Back)

Evaluation was briefly routed through OpenRouter's free Nemotron 30B model. The premise was attractive — batched evaluation would become effectively zero-cost.

End-to-end testing surfaced two hard limits in the free tier: 50 requests / day and 16 / minute. Both are far below what a single school-day's traffic produces — a busy school easily generates several hundred evaluation items per day.

Evaluation went back to Claude Haiku. **OpenRouterLLMClient** is preserved in the repository for a future possibility of paid OpenRouter credits. Walking away from working code wasn't satisfying, but shipping a "zero-cost evaluation" claim that would have broken during exam season — exactly when evaluation traffic spikes — would have been a latent production incident.

5. End-to-End Workflow

When a student asks a question about an uploaded PDF:

1. **Authenticate and rate-limit.** JWT dependency validates the token; role gate + per-feature rate limiter (60 general / 20 AI per minute, plus admin-controlled per-day quota) decide whether the request proceeds.
2. **Ensure PDF is indexed.** ChromaDB collection check; if empty, run extraction → normalization → metadata regex → chunking → embedding → insert.
3. **Retrieve.** Question rewritten into up to 4 variants, batched embedding, top-3k by cosine, rerank with six metadata boosts capped at 1.0.
4. **Grounding check.** Max / avg / term-match / coverage / per-chunk-overlap signals tested against intent-specific thresholds. Failure → fixed refusal sentence, no LLM call.
5. **Build prompt.** Static prohibitions + intent-conditional instruction + audience-conditional budget + top-4 chunks (700 chars each, citation tags) + up to 4 conversation turns.
6. **Call model via LLMClient.** **FallbackLLMClient** checks primary cooldown. If hot, request goes to Claude Sonnet with prompt-caching headers. On failure, set 60s cooldown and retry silently on Ollama.
7. **Validate and return.** Output post-processed (citation extraction, sentence-boundary trimming, multi-stage parsing for quizzes), audit logged, returned to Streamlit.

8. **Background quality scoring.** Admin dashboard periodically launches an LLM-as-judge batch over recent Q&A / summaries / quizzes on partial-credit rubrics. No static golden set — real traffic is the only quality signal that survives contact with production.

Every parameter in this paper is grounded in code under **backend/** and reproducible from the repository.